

MetropoLive

Milestone 1 Report

Group 17 - Sunsets

Yuginda Ranawaka, Tyrese Temple, Sumin Eun, Jacob Yan

<https://github.com/CMPT-276-SUMMER-2025/final-project-17-sunsets>

The 2 APIs:

Google Maps Platform - Maps JavaScript API:

Description: Maps JavaScript is the familiar Google Map panel that can be placed within a website. It allows for several customization options, from map styles, markers and lines, and even the user's viewport. Maps JavaScript will be a foundational part of our app's visuals, and should provide users with an overview of the optimal route between their location and a chosen event.

Google Maps Platform - Routes API:

Description: Routes API is Google's next-generation routing engine, designed to provide high-performance, real-time trip data. Based on secure HTTPS requests, Routes API takes the core functionality of its predecessor, Directions API, and enhances it with more granular data, additional features, and a greater degree of flexibility for developers. At its core, Routes API calculates the optimal path between one or more locations, returning not just a step-by-step set of instructions but also a wealth of contextual information like real-time traffic and multimodal transit options.

Meetup - GraphQL API:

Description: Meetup's GraphQL API will allow our web-app to access a database of live events for a desired area. It gives developers the ability to programmatically search for and retrieve detailed information about scheduled public events, which can range from concerts, workshops, and even market gatherings. This API will be central to providing users with real-time data of the venues they want to access, as well as information that can be translated into dynamic navigation instructions.

Features for each API (6 total):

Google Maps Platform - Maps JavaScript API:

Feature 1 - Map Types: this feature concerns the different types of maps that can be displayed using the Maps JavaScript API. There are four basic types: roadmap, satellite, hybrid, and terrain. Each of these map types displays a different overlay onto our site's primary map panel, the most relevant of which is the roadmap type. Roadmap is specifically designed to make roads, eco-friendly transit routes, and important locations easier to distinguish from their surroundings, which will make real-time navigation with our app more intuitive.

Feature 2 - Markers: including the markers feature will allow users to better identify key locations on a map, similar to dropping a pin. These can include their start and end points, along with any nearby restaurants or local businesses their route passes by. Additionally, the advanced markers option gives us the option to customize the appearance and accessibility of different markers, and even create entirely new markers through HTML and CSS. Document Object Model (DOM) click events can also be configured to trigger unique marker reactions, such as providing a point-form address and venue summary for a chosen event.

Feature 3 - UI Controls: the Maps JavaScript API contains many UI elements that allow users to interact with the map display. These control elements can be enabled/disabled as needed to support different use cases, some of which include zoom control, street view control, and keyboard shortcuts. Depending on our web application wireframing, the default map display can be increased in size from its default 200x200px framing. Integrating these controls will give our users the freedom to manipulate the map view to their needs, which can be especially useful when navigating complicated or unfamiliar environments.

Google Maps Platform - Routes API:

Feature 1 - Compute Routes: this is one of the two core features of Routes API, and allows users to get directions for different modes of travel, and applies to both single or multiple destinations. Compute Routes accommodates several transit forms, including driving, two-wheeled vehicles, walking, or different forms of public transit. Locations can be specified as simple text strings (e.g., "SFU Burnaby Campus"), place IDs, or even longitude and latitude coordinates. Further, the Routes API is designed to be traffic-aware and can utilize real-time traffic data to provide accurate and dynamic routing that may be necessary within high-traffic settings like concerts or festivals.

Feature 2 - Compute Route Matrix: similar to Compute Routes, this feature of Routes API returns the distances and travel times for a given set of origin and destination points held within a matrix. Compute Routes Matrix supports up to 625 distinct route elements, and can allow users to get a visual idea of what several trips to neighboring events would look like, and from this what the most efficient routes would be.

Feature 3 - Request Route Polyline: this feature provides options for the quality of polyline that is returned from a specified method. A basic polyline (default) contains route information without embedded traffic data. Alternatively, traffic-aware polylines are constructed with information about relevant traffic conditions along a route, and can be expressed in terms of different speed settings (e.g., `NORMAL`, `SLOW`, `TRAFFIC\_JAM`). Although of lesser importance when compared to the first two features, this API component can provide additional value to users that need higher fidelity information or greater polyline clarity (via `polylineQuality` for example) prior to leaving for an event.

Meetup - GraphQL API:

Feature 1 - Search Events: events can be specified by location using the `keywordSearch()` method, which will give our users the ability to discover events closest to them. The returned information can be filtered and formatted as needed, and will form the basis of each event card in our listing panel. Having easy access to such data will be crucial for our users to determine what kinds of events are best for their needs or preferences.

Feature 2 - Event Images: this API provides an option to access photos associated with events, and also different groups. This kind of information can be used via a `PhotoInfo` object, which is a flexibility container that supports dimension modifications and file formats like WEBP and PNG. Although a less important feature, presenting users with an image that describes the visual aspects of a local event may make it more appealing to first-time attendees.

Feature 3 - Key Event Information: implementing this feature provides users with a summary of the most important details for the event they plan to attend. Such information could be integrated into several event cards that users can scroll through, where further details such as location, crowd sizes, nearby amenities, and more are listed. It is absolutely crucial to make this type of information both easily accessible and possibly savable for users, as it may be necessary to reference at times while in transit or when coordinating with friends.

Convert low-fidelity storyboard to mid-fidelity prototype:

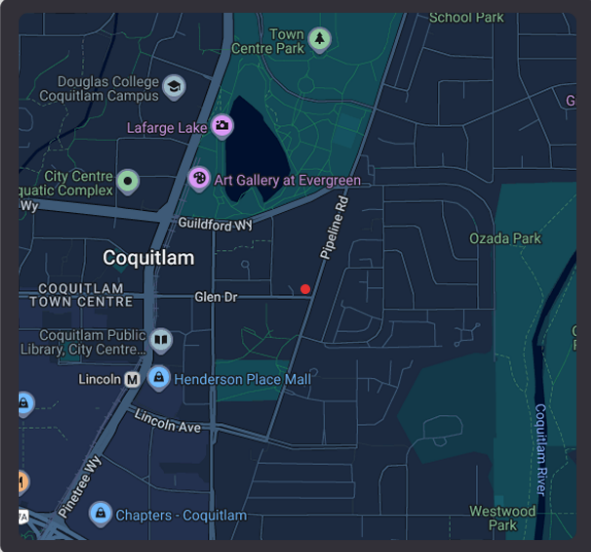
MetropoLive

Enter Location

Please enter your location

MetropoLive

Enter Location



Category:

Radius: KM

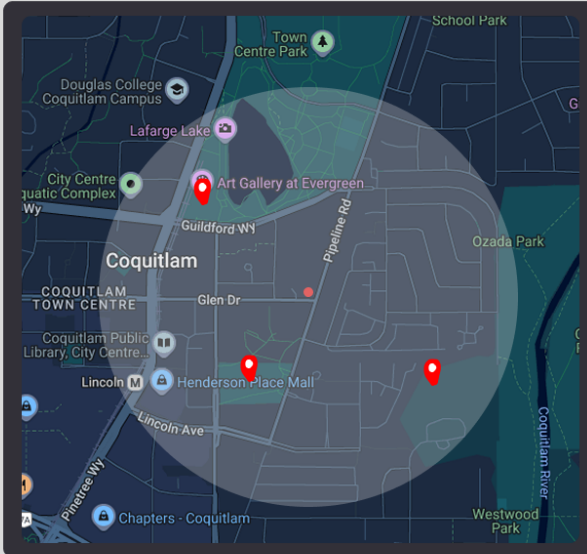
Date: From / / To / /

Price: Min: \$ to Max: \$

Search

MetropoLive

Enter Location



Event name - Location

Date: XX/XX/XXX Price: ##

Description -----

Event name - Location

Date: XX/XX/XXX Price: ##

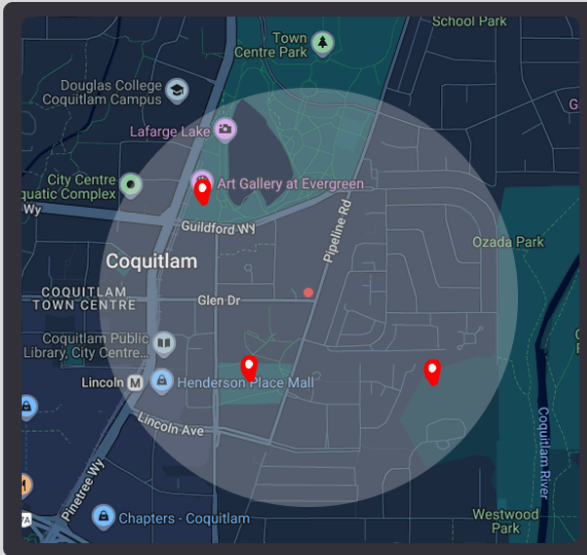
Description -----

Event name - Location

Date: XX/XX/XXX Price: ##

MetropoLive

Enter Location



Event Banner

Event name: -----

Location: -----

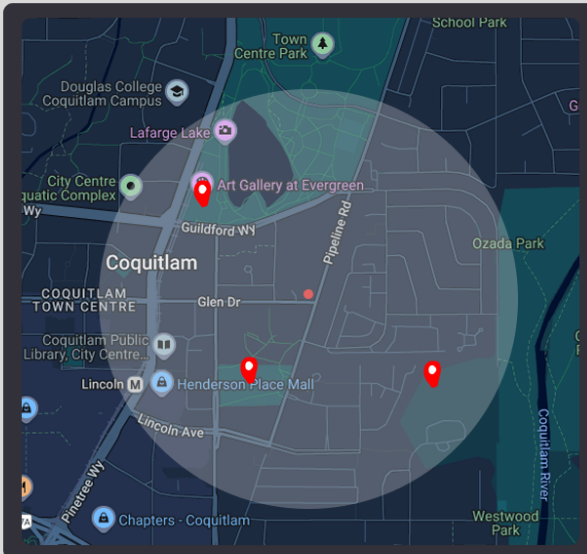
Date: XX/XX/XXXX

Price: ##

Event description: -----

MetropoLive

Enter Location



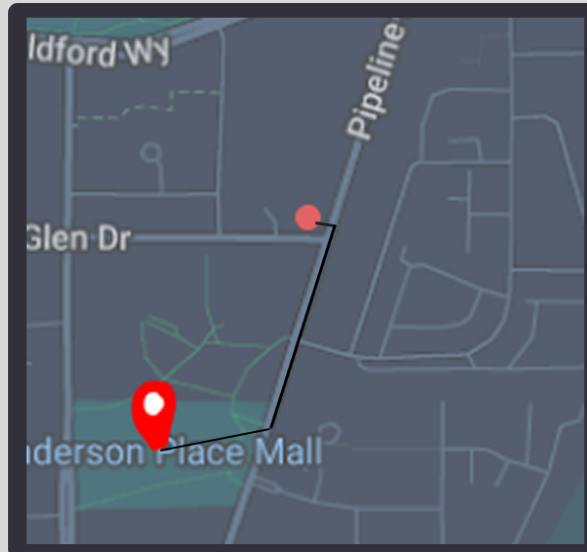
Event description: -----

Search route to event



MetropoLive

Enter Location



Directions:

- Start location
- Walk for #
- Destination

Choose a SDLC model:

For our final project, our team has chosen to use the Agile Kanban SDLC model to structure how we work. The lightweight framework avoids the potentially restrictive, role-based ceremonies of other Agile methods, which will allow each of us to remain adaptable throughout the course of our web application's development. At its core, Kanban's signboard system will make it very simple to obtain a detailed overview of the work that has been completed, or tasks that need extra attention. Taking Github Projects as an example, the visual nature of issue cards placed within different status lanes also increases the visual clarity of our combined workflows without being overwhelming. Each of our group members lead uniquely busy lives with time dedicated to tasks that may not always be within the scope of our web application. Using Kanban will eliminate the need for holding cyclical and iterative collaboration periods such as those found within other frameworks like Agile Scrum. Overall, Kanban is highly supportive of the dynamic and shifting needs of a small development team, all while creating a space that makes it effortless to maintain communication of important tasks or changes throughout each section of the development cycle.

Develop a work breakdown structure of all the tasks:

Project Name: MetroLive		
Level	WBS	Task Description
1	1	Milestone 0 - Resubmission
2	1.1	Add detail to user personas
2	1.2	Improve low-fidelity storyboard
2	1.3	Update tech stack description
1	2	Milestone 1 - Report
2	2.1	Finalize 2 APIs
3	2.1.1	TA approval
3	2.1.2	API descriptions and use cases
3	2.1.3	API features (3 each, 6 total)
4	2.1.3.1	Feature description and user benefit
2	2.2	Mid-fidelity prototype
2	2.3	Choose a SDLC model
3	2.3.1	Brief explanation why it was chosen
2	2.4	Project schedule
3	2.4.1	Timeline of development and testing
3	2.4.2	Milestones and deadlines
3	2.4.3	Recommended: internal deadlines with buffer days
2	2.5	Risk assessment
3	2.5.1	At least 5 low risk issues
4	2.5.1.1	Mitigation strategies
3	2.5.2	At least 5 medium risk issues
4	2.5.2.1	Mitigation strategies
3	2.5.3	At least 5 high risk issues
4	2.5.3.1	Mitigation strategies
2	2.6	Data flow diagrams

Project Name: MetroPoLive		
3	2.6.1	Level 0 diagram
3	2.6.2	Level 1 diagram
2	2.7	MVC model diagram
2	2.8	Appendix
3	2.8.1	List of members and contributions
3	2.8.2	Milestone 0 proposal changelog
3	2.8.3	Any additional diagrams/charts/tables
1	3	Milestone 1 - Video Presentation
2	3.1	Powerpoint slides
2	3.2	Script
3	3.2.1	Overview of project
3	3.2.2	API explanation
3	3.2.3	Prototype walkthrough
3	3.2.4	Overview of application data flow
2	3.3	Video recording
1	4	Website layout
2	4.1	Header
3	4.1.1	Website title/logo (if we have one)
3	4.1.2	Search bar
2	4.2	Map section (map itself not important until Maps API is integrated)
2	4.3	Instructions
2	4.4	Information form
3	4.4.1	Labels
3	4.4.2	User input fields
3	4.4.3	Radius slider
3	4.4.4	Search button
2	4.5	Event listings
3	4.5.1	Info for each event

Project Name: MetroLive		
3	4.5.2	Separate box for each event
2	4.6	Event details
3	4.6.1	Event banner
3	4.6.2	Detailed event information
3	4.6.3	Search route to event
2	4.7	Directions
3	4.7.1	Walking/driving/transit/biking buttons
3	4.7.2	Step-by-step directions (again, likely not important until API integration)
2	4.8	Style
3	4.8.1	Colors
3	4.8.2	Fonts
1	5	API integration
2	5.1	Google Maps API
2	5.2	Meetup API
1	6	Functionality/interactivity
2	6.1	User input
3	6.1.1	Location search bar
3	6.1.2	Information form
3	6.1.3	Radius slider
3	6.1.4	Search button
3	6.1.5	Clickable events (expands event info)
3	6.1.6	Search route to event
3	6.1.7	Switch between transportation methods
2	6.2	Map updates from Routes API
3	6.2.1	Show location
3	6.2.2	Show nearby events
3	6.2.3	Show route
2	6.3	Display event information from Meetup API

Project Name: MetroLive

1	7	Testing
1	8	Milestone 1.5 - Check-in
2	8.1	1 page report (progress on following items)
3	8.1.1	API 1 features (completed or near completion)
3	8.1.2	API 2 features (confirmed or planned)
3	8.1.3	Application interface
4	8.1.3.1	Developed and interactive, includes API 1 features
3	8.1.4	CI/CD
3	8.1.5	Application testing
4	8.1.5.1	Automated tests for API 1 features
2	8.2	15 minute meeting with TA

Project schedule with milestones and deadlines:

[Project Schedule](#)

Risk assessment of potential issues and how to mitigate them:

Low-Risk Issues

1. **Minor UI/UX Inconsistencies:** Depending on the user's web browser (e.g., Chrome, Firefox, Safari), our application's structuring may be displayed with slight variance, which could reduce the usability of certain features. Mitigation Strategy: We could use a CSS reset stylesheet at the beginning of our project to standardize the ways in which different browsers apply built-in styling to different HTML elements. This is likely the only action required, but it may also be desirable to use a styling framework such as Tailwind CSS to handle cross-browser compatibility automatically. To confirm the efficacy of these mitigation steps, we could test our web application within several different browsers.
2. **Infrequent API Endpoint Changes:** One of our API providers unexpectedly depreciates a minor endpoint that our app uses for non-critical features. Mitigation Strategy: Implement logging of API calls and their responses, and set up an alert system that looks for specific HTTP(S) error codes that correspond to an API endpoint not functioning as intended. We would have to review these logs regularly, or make it so that important error codes automatically send all team members a message via email for example.
3. **Ineffective Map Markers:** The markers used to define both the user's and available event locations are unclear, or don't load in as anticipated. Mitigation Strategy: We could create much simpler marker alternatives, as Maps JavaScript API supports custom HTML/CSS marker icons. These could either be used in place of the default markers entirely, or be used as a debugging tool that helps us determine what is not being displayed as intended and why.
4. **Accessibility Compliance Gaps:** Our application's color scheme, font font sizes, or interactive elements do not align with modern accessibility standards. Mitigation Strategy: Disabilities such as color blindness are very important accessibility factors to consider prior to starting development. As such, our application's design should include the most common accessibility checks into the interface. Additionally, users would be given the freedom to interact with every core feature of our web application via their keyboard, instead of just a mouse or trackpad.
5. **Deprecation of a Third-Party Library:** A JS library that's used for a non-critical feature like formatting becomes outdated and is dropped from receiving future updates or community support. Mitigation Strategy: Here, we would have to find a popular alternative that works for our use case, specifically one with an active support team or otherwise community behind it. Once secured, the team would have to hold a review of all dependencies that our application uses, and possibly

maintain a list of usable alternatives that can be quickly used should anything lose support.

Medium-Risk Issues

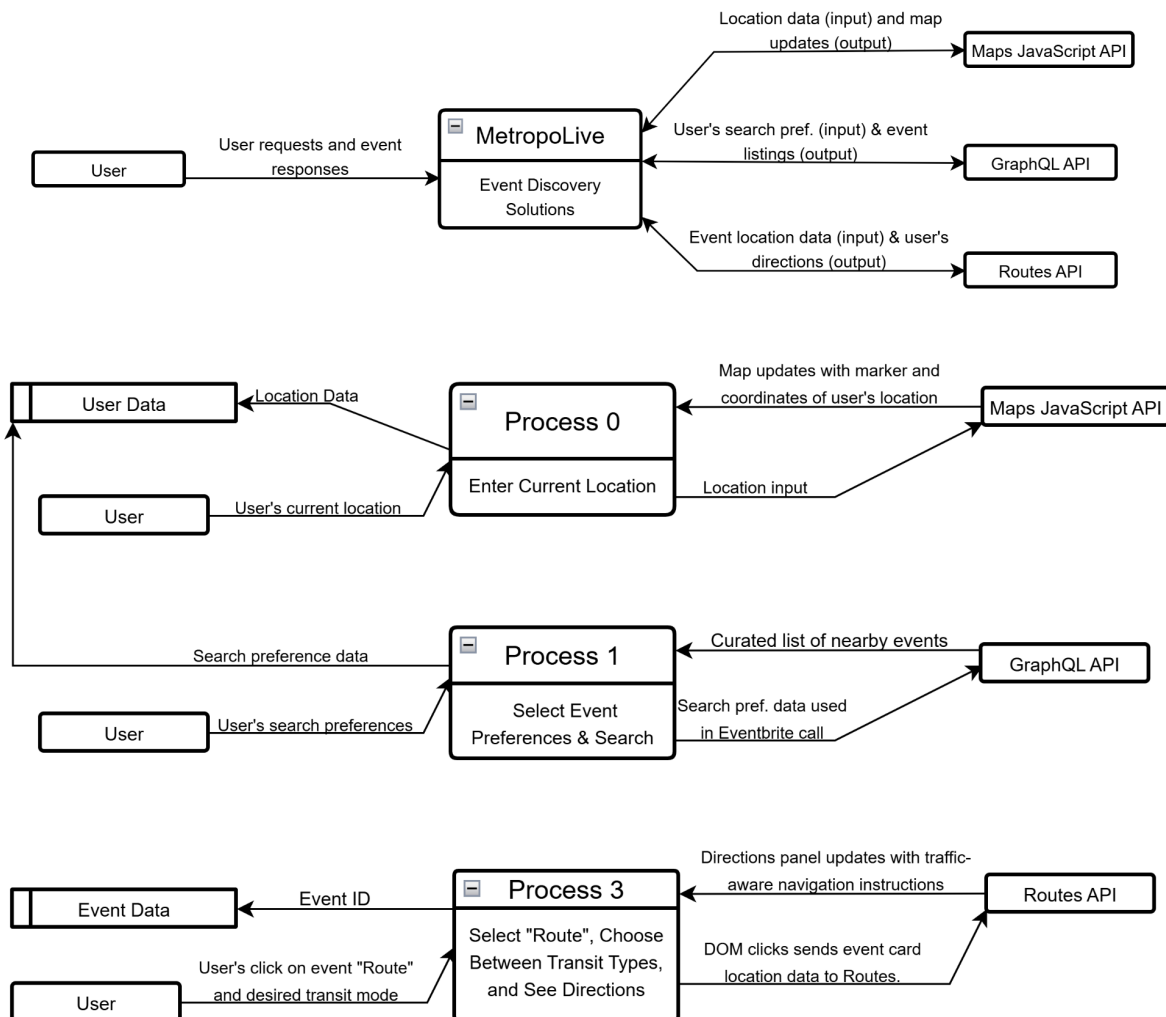
1. **Unexpected API Usage Costs:** A sudden increase in user traffic that leads to an unexpected jump in API calls, resulting in costs that exceed the project's budget or the free tier limits (our case). Mitigation Strategy: Given that this application should stay within the bounds of each API's respective number of free calls, it's recommended to implement three separate call counters that notify all members of the development team of the remaining API calls for our application. In addition to this, we would have to define strict budget alerts in the event that our API usage must incur a small number of overage fees.
2. **Inconsistent or Incomplete API Data:** API return data that can't be presented to the user without careful modification. Mitigation Strategy: Develop a robust data validation layer on the application's backend. This layer should format and sanitize all incoming data, removing any unwanted information, and substitute sensible defaults for missing or null fields (e.g., "Event address not available" instead of "NULL").
3. **Hitting API Rate Limits:** Sending too many API calls back to, for example, Google Maps Platform could cause the maps panel to be temporarily throttled or otherwise blocked, which could limit user experience. Mitigation Strategy: On the backend, we could implement a caching system for API responses to avoid asking for information that was already sent earlier. Additionally, failed requests could be held within a queue that works by exponential backoff logic (see resource). Here, the amount of time between each request retry increases based on the number of failed requests built up in the queue. Resource: <https://cloud.google.com/memorystore/docs/redis/exponential-backoff#:~:text=Exponential%20backoff%20is%20a%20standard,with%20increasing%20delays%20between%20requests>.
4. **Inaccurate User Geolocation Input:** The user's provided location is inaccurate, and can't be displayed on the map. Mitigation Strategy: Design a graceful fallback system that's based on proper usability heuristics. This way, the user is notified of their input error and the limited functionality that app features may have because of this. Also, re-prompt the user for another "Current Location" if they acknowledge the previous information.
5. **Directions Don't Meet User Needs:** The directions provided by the Google Routes API do not account for specific user needs, such as finding wheelchair-accessible routes or avoiding toll roads. Mitigation Strategy: Place the primary focus on the core transit modes (e.g., driving, walking, public transit) for our initial launch. Following this, provide a clear timeline of development features that include greater navigation-related accessibility considerations, and outline when such features should be available to users.

High-Risk Issues

1. **Major Breaking API Change:** For example, Google or Meetup send out a required update to their respective APIs that's not backward-compatible, which causes core features of our map (e.g., displaying maps or searching for events within the user's location) to stop working or function improperly. Mitigation Strategy: Backend team members should stay up to date on all official developer release channels for our APIs. This may include reading blog articles, email, or even company newsletters that outline upcoming changes to their API services, and how this may impact our application's usability. Further, it may also be possible to implement a background service that attempts basic use of our app, and reports if an error/exception was encountered anywhere in this process (a bit more involved).
2. **Client-Side Exposure of API Keys:** The API keys for any one of our features are accidentally embedded in the client-facing JS code. Mitigation Strategy: This is an important consideration as leaving such a mistake in our application may lead to unintended abuse of our APIs. To mitigate this, we could set up a simple proxy server using a service like Google Cloud Functions or Netlify, where our API keys are stored instead of within the client-side JavaScript. API calls would then be made to the URL that points to our proxy server, thus stopping a user from inspecting our JS and stealing any keys.
3. **Unable to Adhere to Development Schedule:** The implementation of certain features takes longer than the time allotted to them, thus reducing the amount of attention that can be given to other features/issues. Mitigation Strategy: Similar to Agile Scrum, take notice of team communication surrounding persistent issues, and make sure to bring these up in ad hoc meetings. As a group, identify the biggest challenges preventing the implementation of a fast solution to each of the issues, and formulate a short-term strategy to resolve said issues while still devoting time to upcoming tasks.
4. **Violation of API Terms of Service (ToS):** Our web application fails to comply with one of the API provider's usage policies, such as being required to display "Powered by Google" logo on the map panel, which leads to abrupt suspension of API access. Mitigation Strategy: Prior to starting the implementation phase, each of the frontend and backend team members should thoroughly review any relevant usage policies created by our API providers. From this, we could create a compliance checklist that can be referenced at any time within deployment. Any subsequent visual changes that may need to be added into our implementation plan should also be given sufficient time to design.
5. **Team Member Leaves Due to Exigent Circumstances:** Due to unforeseen circumstances, one or more members of our team is required to leave temporarily (or even permanently) during development. Mitigation Strategy: While not necessarily something that can be controlled, each phase of our development life cycle should take into account the amount of time one or more team members may need to step away from their work. As well, brief but detailed implementation logs should be kept by each team member, which are separate from basic comments within code. These files may reduce the amount of

operational overhead required should a new team member need to be substituted in at any point.

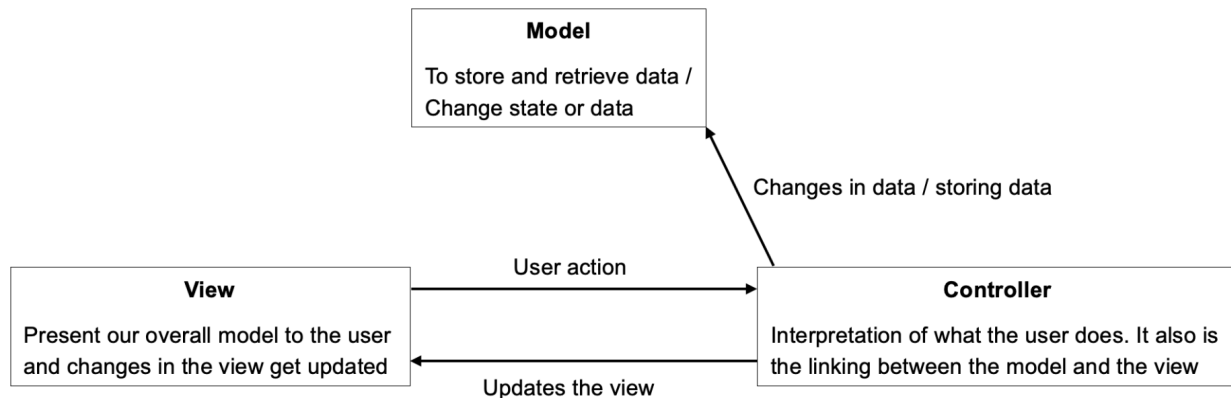
Data Flow Diagrams (level 0 and level 1) to outline how data flows within the application:



The data flow diagrams' above show how the user inputs (which are shown on the left most side) such as location data and search preferences are processed individually within our web application and interact with our selected APIs. For instance, process 0 shows that user inputs on location data will be processed and will retrieve data from the Maps JavaScript API. In process 1 Meetup's API will retrieve a list of nearby events when a user makes a search and process 3 which provides navigation once Routes API

is interacted with. In summary the above data flow diagrams show the process in which user input connects with the APIs above within MetroPoLive.

MVC model diagram for application:



- 1. Model:** The model is responsible for storing the list of events, which we retrieve from the Meetup API using the fetch function within react itself. It keeps track of a particular route that a user is interested in by integrating data from the Google Maps API, while handling the search filters and user preferences as required. Moreover, the model oversees maintaining the state of our web application, thus enabling updates with regards to events or route changes to be reflected on the web application as soon as possible. In summary the model includes the process of storing data of the events, storing the input that a user declares and keeping track of route data of users.
- 2. Controller:** The process that links the users' actions to a particular data change is what the controller oversees. For instance, when our users search for events, interact with the maps or decide on routes to choose on the web application the controller will make use of these user actions by using react event handlers (for example OnClick handler) to make API calls and to reflect updates on the model. Following the updated model the controller will note the view to update the changes. In summary the controller will use fetch to make API calls when users make an input, once the state is updated it will make updates to the react components like a component which we will create called maps.
- 3. View:** The elements of the user interface that we will build using react is what makes up the view. For instance, the navigation bar, event cards, map, search bar and the other required components will make up the view in our web application. The way our data is presented to the user from the model is what the view is responsible for. For example, our website should be visually pleasing and interactive to users. Further when the data from the model changes and updates

are made the view is immediately re-rendered to reflect the updated information.
In summary the view will include the components in which the user interacts with.

Appendix of any additional information that was not included in main report:

Tyrese Temple: collected API research for both the primary and secondary sets, pricing information, and desired features to implement. Created the DFD diagrams (level 0 and 1). Wrote the SDLC explanation & risk assessment.

Sumin Eun: Project manager, created outlines of documents, contributed to the project proposal by writing the overview and general start of the document, contributed to writing the personas and users, drew out all storyboards and interfaces.

Jacob Yan: Contributed to user personas and user stories and some API descriptions in the project proposal, created GitHub issues, WBS, and project schedule for current tasks.

Yuginda Ranawaka: Created the model view controller and contributed to the project proposal by writing up the MVC details and DFD description, cleaned up the technology stack to suit our APIs. Contributed to GitHub issues.

Changelog of revisions since proposal (based on feedback):

Feedback:	Changes made:	Edited by:	Date:
Personas need more details	Added categories with way more personal details as suggested by the comments.	Tyrese	June 27
Updated API	Switched to Meetup GraphQL API instead of Eventbrite due to their location-based search endpoint being deprecated since 2019.	Tyrese	July 3
Low fidelity storyboard needs to be more visible	Cleaned up low-fidelity storyboard	Sumin	June 17
Be more descriptive with tech stack	Changed explanation to show	Yuginda	July 4

usage	steps of how we are going to use the tech stack to build our app.		
Prioritize core features	Placing greater focus on first implementing (1) the map view with Maps JavaScript API, and (2) providing a curated event list with Meetup GraphQL API.	Tyrese	July 4